

Report

Part 1:

In this part, I was set to define a drift test detection. I have defined a separate flow using Metaflow, named `drift_detection_ks`. It accepts two datasets, here in parquet format and using **KS** metric, it will calculate the drift for any feature in the vector of inputs.

note: to speed up this step, I used the test split part of the main dataset as the reference.

In this test, I'm checking whether there is a drift in features or not. with respect to the above note, [2] image depicts the result. Here I'm seen drift in four features, so to interpret this I used gpt-4o, its the result:

When the report says there is **drift in 4 features**, it means:

The data distribution for 4 specific features has changed significantly compared to the reference dataset.

This implies:

- These 4 features — for example `desc_tfidf__include`, `desc_tfidf__lead`, etc. — now follow a different pattern than before.
- This change is **statistically significant**, based on a KS test with $p < 0.05$.

Why it matters:

- Your model may **not perform as well** if it relies on these features.
- It could signal:
 - A **change in user behavior**
 - An **upstream bug** or preprocessing change
 - A **shift in data source** or content
 - Natural **concept drift** (e.g., job descriptions evolving over time)

What you might do:

- Investigate *why* these features drifted.
- Retrain the model if necessary.
- Monitor the drift over time.

In summary: **4 features changed in a way that might impact your model's reliability.**

Part 2:

Ok, for this step since I have used Metaflow, I have used Parameters previously so then I can pass multiple parameters to train my model. Additionally to benefit for more reusability, I have used mixins for dataset, model and experiment so then I can have different flows and share this between them while my training part is different.

for further details, have a look at [experiment_1_decision_tree_fault_tolerance](#).

Part 3:

For comparing the results of evaluation in the last step of the A/B test, I'm using Mean Absolute Error, which in my salary prediction is the absolute difference between ground truth and the predicted salary, mean of all data. In this scenario, the model with less MAE outperforms the other one.

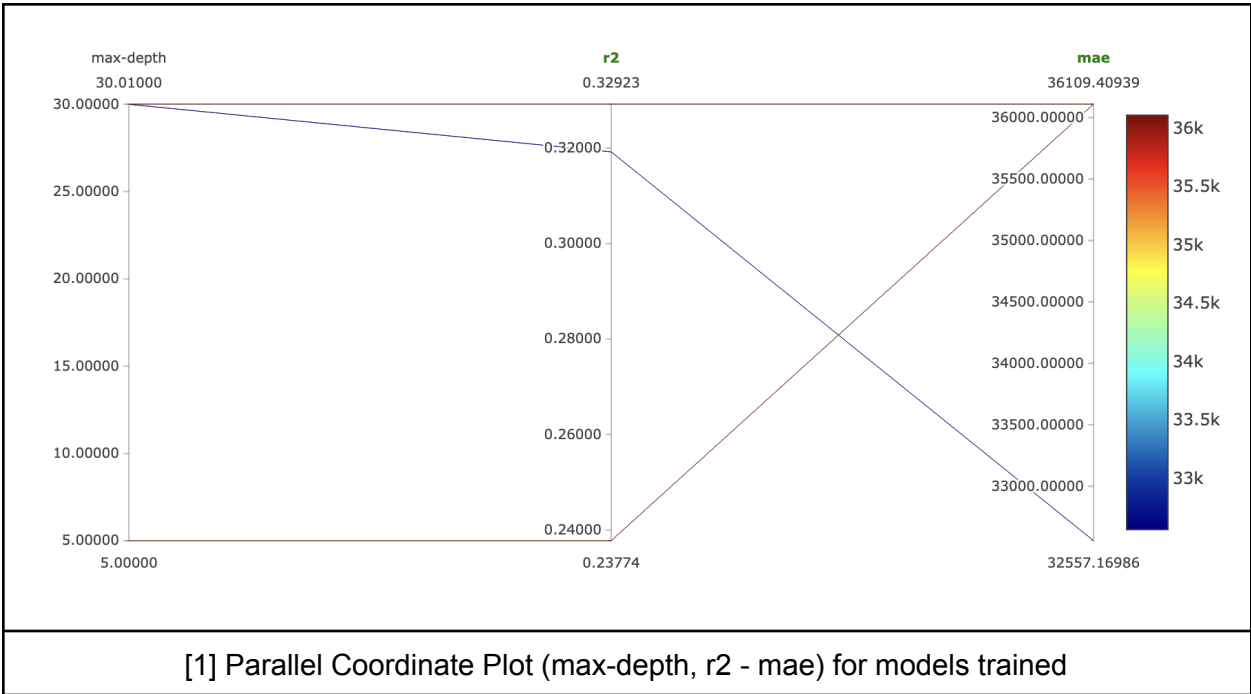
For A/B testing, I have updated my training flow. It saves the preprocessor as an artifact to the experiment run. Basically, flow called [evaluate_A_B.py](#) accepts two parameters, model uri from MLFlow and the dataset.

It validates the dataset, and loads the preprocessor for each model, for being consistent. (I have faced multiple issues due to lack of knowledge in basic ML, for example check logs attachments in the submission).

Then for each model, I run the predict on the dataset. for result check the A/B step in the [offline_evaluation.ipynb](#) notebook.

In addition to it, I have used gpt-4o to customize the previous pipeline. Instead of two models, it accepts an experiment-name and loads all the models in this and predicts with them. It is named [evaluate_experiment.py](#).

Appendix:



Drift Detection Report				
Total features checked: 14421				
Features with drift (p < 0.05): 4				
Features with Drift				
Feature	KS Stat	P-Value	Drift Detected	
desc_tfidf__include	0.0198	0.0497	Yes	
desc_tfidf__lead	0.0211	0.03	Yes	
desc_tfidf__office	0.0205	0.0378	Yes	
desc_tfidf__systems	0.02	0.0467	Yes	
Drift detected in 4 of 14421 features.				
[2] Drift detection flow output result				